

07 — Schema Design: SaaS Multi-Tenant Patterns

Table of Contents

- 1. [Learning Objectives](#)
- 2. [Multi-Tenancy Overview](#)
- 3. [Pattern 1: Shared Database, Shared Schema \(Row-Level Security\)](#)
- 4. [Pattern 2: Shared Database, Schema-Per-Tenant](#)
- 5. [Pattern 3: Database-Per-Tenant](#)
- 6. [Pattern Comparison Matrix](#)
- 7. [Complete RLS-Based SaaS Schema](#)
- 8. [Complete Schema-Per-Tenant Implementation](#)
- 9. [Hybrid Patterns](#)
- 10. [Common Queries & Operations](#)
- 11. [Common Mistakes](#)
- 12. [Best Practices](#)
- 13. [Performance Considerations](#)
- 14. [Interview Questions & Answers](#)
- 15. [Exercises with Solutions](#)
- 16. [Cross-References](#)

Learning Objectives

After completing this section you will be able to:

- Choose the right multi-tenant strategy for a given SaaS application
- Implement Row-Level Security (RLS) for tenant isolation
- Design schema-per-tenant with automated provisioning
- Handle tenant onboarding, offboarding, and data isolation
- Understand the performance and operational trade-offs of each pattern

Multi-Tenancy Overview

Multi-tenancy means multiple customers (tenants) share the same database infrastructure, with their data isolated from each other.

Three main strategies:

Strategy	Database	Schema	Tables	Isolation Level
Shared Schema	Shared	Shared	Shared	Row-level (RLS)
Schema-Per-Tenant	Shared	Separate	Per-tenant	Schema-level
DB-Per-Tenant	Separate	Separate	Per-tenant	Database-level

Number of tenants:

- < 50 tenants: DB-per-tenant may be practical
- 50-10K tenants: Schema-per-tenant is sweet spot
- > 10K tenants: Shared schema (RLS) scales best

Pattern 1: Shared Database, Shared Schema

All tenants share the same tables. A `tenant_id` column identifies each row's owner. Row-Level Security (RLS) automatically filters data by tenant.

One set of tables:

Table: projects			
project_id	tenant_id	name	status
1	acme	Website	active
2	globex	App	active
3	acme	Mobile App	planning

← same table!

RLS policy: `WHERE tenant_id = current_setting('app.tenant_id')`

Pros and Cons

Pros:

- ✓ Simple schema management (one set of migrations)
- ✓ Scales to thousands/millions of tenants
- ✓ Efficient resource usage (shared connections, shared cache)
- ✓ Cross-tenant reporting is simple (admin bypasses RLS)
- ✓ No catalog bloat (not thousands of schemas)

Cons:

- ✗ RLS misconfiguration = data leak
- ✗ Noisy neighbor problem (large tenant affects others)
- ✗ Per-tenant customization is difficult
- ✗ Regulatory isolation requirements may not be met (GDPR, HIPAA)
- ✗ All tenant data in one backup

Pattern 2: Shared Database, Schema-Per-Tenant

Each tenant gets their own PostgreSQL schema with identical table structures. Data is physically separated by schema.

One database, many schemas:

```
Database: saas_app
├─ Schema: tenant_acme
│   ├── projects
│   ├── tasks
│   └─ users
├─ Schema: tenant_globex
│   ├── projects
│   ├── tasks
│   └─ users
└─ Schema: public (shared metadata)
```

```
├─ tenants
├─ tenant_configs
```

Pros and Cons

- Pros:
- ✓ Strong logical isolation (separate namespaces)
 - ✓ Easy per-tenant restore (pg_dump one schema)
 - ✓ Per-tenant customization possible
 - ✓ No RLS configuration risk
 - ✓ Easier compliance isolation
- Cons:
- ✗ Schema management complexity (migrations run N times)
 - ✗ Catalog bloat at >1000 tenants (pg_class grows)
 - ✗ Cross-tenant reporting requires UNION or FDW
 - ✗ Connection overhead (pgbouncer pool fragmentation)

Pattern 3: Database-Per-Tenant

Each tenant gets their own PostgreSQL database.

- Pros:
- ✓ Maximum isolation (separate databases = separate processes possible)
 - ✓ Easy per-tenant backup, restore, migrate
 - ✓ Compliance requirements easily met
 - ✓ Custom extensions per tenant
- Cons:
- ✗ Very high operational overhead at scale
 - ✗ Connection management is complex (separate connection pools)
 - ✗ Expensive (resources not shared)
 - ✗ Only feasible for small number of tenants (< 50)

Pattern Comparison Matrix

	Shared Schema	Schema/Tenant	DB/Tenant
Max tenants	Millions	~5,000	~50
Isolation strength	Row (RLS)	Schema	Database
Migration effort	Simple	Moderate	Complex
Per-tenant backup	Difficult	Easy (dump)	Trivial
Cross-tenant query	Easy (admin)	UNION/FDW	FDW only
Catalog size	Small	Grows with N	N databases
Connection pooling	Efficient	Moderate	Difficult
Compliance	Hard for strict	Medium	Strong
Customization	Hard	Easy	Easy
Operational cost	Low	Medium	High

Best for	B2C, many small tenants	Medium B2B teams	Enterprise/VIP
----------	-------------------------	------------------	----------------

Complete RLS-Based SaaS Schema

```
-- =====
-- SAAS PLATFORM – SHARED SCHEMA WITH ROW-LEVEL SECURITY
-- =====

-- =====
-- TENANT MANAGEMENT (in public schema)
-- =====

CREATE TABLE tenants (
    tenant_id      UUID          NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
    slug           TEXT          NOT NULL UNIQUE CHECK (slug ~ '^[a-z][a-z0-9\-\]{1,62}$'),
    name           TEXT          NOT NULL,
    plan           TEXT          NOT NULL DEFAULT 'starter'
                        CHECK (plan IN
('starter','growth','professional','enterprise')),
    status         TEXT          NOT NULL DEFAULT 'active'
                        CHECK (status IN
('active','suspended','cancelled','trial')),
    trial_ends_at  TIMESTAMPTZ,
    max_users      INTEGER       NOT NULL DEFAULT 5,
    max_projects   INTEGER       NOT NULL DEFAULT 10,
    created_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at     TIMESTAMPTZ   NOT NULL DEFAULT now()
);

CREATE TABLE tenant_users (
    id             BIGINT        GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tenant_id      UUID          NOT NULL REFERENCES tenants(tenant_id) ON DELETE
CASCADE,
    user_id        UUID          NOT NULL, -- references auth.users
    role           TEXT          NOT NULL DEFAULT 'member'
                        CHECK (role IN ('owner','admin','member','viewer')),
    is_active      BOOLEAN       NOT NULL DEFAULT TRUE,
    joined_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, user_id)
);

-- =====
-- APP TABLES (with tenant_id for RLS)
-- =====

CREATE TABLE projects (
    project_id     UUID          NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
    tenant_id      UUID          NOT NULL REFERENCES tenants(tenant_id) ON DELETE
```

```

CASCADE,
    name            TEXT            NOT NULL CHECK (length(trim(name)) >= 1),
    description     TEXT,
    status          TEXT            NOT NULL DEFAULT 'active'
                                CHECK (status IN ('active','archived','deleted')),
    owner_user_id   UUID            NOT NULL,
    settings        JSONB           NOT NULL DEFAULT '{}',
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

CREATE TABLE tasks (
    task_id         UUID            NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
    tenant_id       UUID            NOT NULL REFERENCES tenants(tenant_id) ON DELETE
CASCADE,
    project_id      UUID            NOT NULL REFERENCES projects(project_id) ON DELETE
CASCADE,
    title           TEXT            NOT NULL,
    description     TEXT,
    status          TEXT            NOT NULL DEFAULT 'todo'
                                CHECK (status IN
('todo','in_progress','review','done','cancelled')),
    priority        SMALLINT        NOT NULL DEFAULT 2 CHECK (priority BETWEEN 1 AND 5),
    assignee_user_id UUID,
    due_date        DATE,
    created_by      UUID            NOT NULL,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- Indexes
CREATE INDEX idx_projects_tenant ON projects (tenant_id);
CREATE INDEX idx_tasks_tenant   ON tasks (tenant_id);
CREATE INDEX idx_tasks_project  ON tasks (project_id);
CREATE INDEX idx_tasks_assignee ON tasks (assignee_user_id, tenant_id);

-- =====
-- ROW-LEVEL SECURITY
-- =====

-- Enable RLS on all tenant-scoped tables
ALTER TABLE projects ENABLE ROW LEVEL SECURITY;
ALTER TABLE tasks   ENABLE ROW LEVEL SECURITY;

-- App user role (application connects as this role)
CREATE ROLE saas_app_user;
GRANT USAGE ON SCHEMA public TO saas_app_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON projects, tasks TO saas_app_user;

-- RLS policies: app sets tenant context via a session setting
-- The application must call: SET app.current_tenant_id = '<tenant_uuid>';

```

```

CREATE POLICY tenant_isolation_projects ON projects
  FOR ALL
  TO saas_app_user
  USING (tenant_id = current_setting('app.current_tenant_id', TRUE)::UUID)
  WITH CHECK (tenant_id = current_setting('app.current_tenant_id', TRUE)::UUID);

CREATE POLICY tenant_isolation_tasks ON tasks
  FOR ALL
  TO saas_app_user
  USING (tenant_id = current_setting('app.current_tenant_id', TRUE)::UUID)
  WITH CHECK (tenant_id = current_setting('app.current_tenant_id', TRUE)::UUID);

-- Admin bypass: superuser or admin role can see all tenants
CREATE ROLE saas_admin;
GRANT ALL ON ALL TABLES IN SCHEMA public TO saas_admin;

-- Admin sees all rows (BYPASSRLS or policy exception)
CREATE POLICY admin_bypass_projects ON projects
  FOR ALL
  TO saas_admin
  USING (TRUE); -- no filter for admins

-- Set tenant context function
CREATE OR REPLACE FUNCTION set_tenant_context(p_tenant_id UUID)
RETURNS void LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
  PERFORM set_config('app.current_tenant_id', p_tenant_id::TEXT, TRUE);
END;
$$;

-- Example: application usage
-- SELECT set_tenant_context('acme-uuid-here');
-- SELECT * FROM projects; -- only sees acme's projects
-- INSERT INTO projects (tenant_id, name, owner_user_id) VALUES
(current_setting('app.current_tenant_id')::UUID, 'New Project', '...');

```

Complete Schema-Per-Tenant Implementation

```

-- =====
-- TENANT REGISTRY (public schema)
-- =====

CREATE TABLE tenants (
  tenant_id  UUID      NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
  slug       TEXT      NOT NULL UNIQUE CHECK (slug ~ '^[a-z][a-z0-9_]{1,62}$'),
  name       TEXT      NOT NULL,
  schema_name TEXT     NOT NULL UNIQUE GENERATED ALWAYS AS ('t_' || slug) STORED,
  plan       TEXT      NOT NULL DEFAULT 'starter',
  is_active  BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

);

-- =====
-- TENANT PROVISIONING
-- =====

CREATE OR REPLACE PROCEDURE provision_tenant_schema(p_slug TEXT, p_name TEXT)
LANGUAGE plpgsql AS $$
DECLARE
    v_schema TEXT := 't_' || p_slug;
    v_tenant_id UUID := gen_random_uuid();
BEGIN
    -- Insert tenant record
    INSERT INTO tenants (tenant_id, slug, name) VALUES (v_tenant_id, p_slug,
p_name);

    -- Create schema
    EXECUTE format('CREATE SCHEMA IF NOT EXISTS %I', v_schema);

    -- Create tenant's tables (identical structure, different schema)
    EXECUTE format($$
        CREATE TABLE %I.users (
            user_id        UUID        NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
            email          TEXT        NOT NULL UNIQUE,
            full_name      TEXT        NOT NULL,
            role           TEXT        NOT NULL DEFAULT 'member'
                                CHECK (role IN ('owner', 'admin', 'member', 'viewer')),
            is_active      BOOLEAN NOT NULL DEFAULT TRUE,
            created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
        )
    $$, v_schema);

    EXECUTE format($$
        CREATE TABLE %I.projects (
            project_id     UUID        NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
            name           TEXT        NOT NULL,
            description    TEXT,
            status         TEXT        NOT NULL DEFAULT 'active',
            created_by     UUID        NOT NULL REFERENCES %I.users(user_id),
            created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
        )
    $$, v_schema, v_schema);

    EXECUTE format($$
        CREATE TABLE %I.tasks (
            task_id        UUID        NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
            project_id     UUID        NOT NULL REFERENCES %I.projects(project_id) ON
DELETE CASCADE,
            title          TEXT        NOT NULL,
            status         TEXT        NOT NULL DEFAULT 'todo',
            assignee_id    UUID        REFERENCES %I.users(user_id),
            due_date       DATE,

```

```

        created_by  UUID      NOT NULL REFERENCES %I.users(user_id),
        created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
    )
    $$, v_schema, v_schema, v_schema, v_schema);

-- Create tenant-specific role
EXECUTE format('CREATE ROLE IF NOT EXISTS %I', v_schema || '_role');
EXECUTE format('GRANT USAGE ON SCHEMA %I TO %I', v_schema, v_schema || '_role');
EXECUTE format('GRANT ALL ON ALL TABLES IN SCHEMA %I TO %I', v_schema, v_schema
|| '_role');

RAISE NOTICE 'Tenant % provisioned with schema %', p_name, v_schema;
END;
$$;

-- Provision new tenants
CALL provision_tenant_schema('acme', 'Acme Corp');
CALL provision_tenant_schema('globex', 'Globex Industries');

-- Application sets search_path for each tenant connection:
-- SET search_path = t_acme, public;
-- Then all queries work without schema prefix:
-- SELECT * FROM projects; -- queries t_acme.projects

```

Hybrid Patterns

Core data shared, sensitive data per-schema

```

-- Shared core
CREATE TABLE public.tenants (...);
CREATE TABLE public.plans (...);

-- Sensitive per-tenant schema
-- t_acme.billing_data
-- t_acme.user_pii
-- t_globex.billing_data
-- t_globex.user_pii

```

Hot/cold split

```

-- Hot data (current period) in shared tables with RLS
CREATE TABLE recent_events (tenant_id UUID, ...) -- last 90 days, RLS

-- Cold data (archived) in per-tenant schemas
-- t_acme.events_archive_2023
-- t_globex.events_archive_2023

```

Common Queries & Operations

```
-- Cross-tenant report (admin only, bypasses RLS)
SET ROLE saas_admin; -- or BYPASSRLS attribute
SELECT
    t.name AS tenant,
    count(p.project_id) AS project_count,
    count(tk.task_id) AS task_count
FROM tenants t
LEFT JOIN projects p ON p.tenant_id = t.tenant_id
LEFT JOIN tasks tk ON tk.project_id = p.project_id
GROUP BY t.tenant_id, t.name
ORDER BY project_count DESC;

-- Tenant context in application
BEGIN;
SELECT set_tenant_context('acme-tenant-uuid');
SELECT * FROM projects WHERE status = 'active'; -- only acme's projects
INSERT INTO tasks (tenant_id, project_id, title, created_by)
VALUES (current_setting('app.current_tenant_id')::UUID, 'proj-uuid', 'New Task',
'user-uuid');
COMMIT;

-- Check RLS is working
RESET ROLE;
SET ROLE saas_app_user;
SELECT set_tenant_context('acme-uuid');
SELECT count(*) FROM projects; -- returns only acme count
SELECT count(*) FROM projects WHERE tenant_id !=
current_setting('app.current_tenant_id')::UUID;
-- Should return 0!
```

Common Mistakes

1. **Forgetting `WITH CHECK` in RLS policy** — `USING` controls reads; `WITH CHECK` controls writes. Without `WITH CHECK`, a user can insert rows for other tenants!
2. **Not calling `set_config` before every query** — In a connection pool, connections are reused. If you forget to set the tenant context, you might use the previous session's tenant ID.
3. **Not testing RLS bypass** — Test with the application role that tenants use (not superuser) to verify isolation.
4. **Schema-per-tenant without migration tooling** — Running schema migrations across 1000 tenant schemas manually is dangerous. Use a migration script that iterates over all tenant schemas.
5. **Using schema-per-tenant beyond ~5000 tenants** — `pg_class` grows large, query planning slows down, catalog cache pressure increases.

Best Practices

1. Choose pattern based on tenant count and isolation requirements:

- <50 tenants: DB-per-tenant
- 50–5000 tenants: Schema-per-tenant
- 5000 tenants or unknown scale: Shared schema with RLS

2. Always test RLS policies with a non-superuser role.

3. Set tenant context as the first operation in every database session.

4. Use connection pooling (PgBouncer) with transaction mode for shared schemas.

5. Automate tenant provisioning — never create tenant schemas manually.

6. Include a tenant migration runner in your deployment pipeline.

Performance Considerations

```
-- Shared schema: RLS adds one filter per query
-- With proper indexes on tenant_id, this is negligible:
CREATE INDEX idx_projects_tenant ON projects (tenant_id, created_at DESC);
-- Index-only scan possible for tenant's recent projects

-- Connection overhead per pattern:
-- Shared schema:      1 pool → all tenants (most efficient)
-- Schema-per-tenant:  1 pool per schema (or dynamic search_path)
-- DB-per-tenant:      1 pool per DB (resource-intensive)

-- Schema-per-tenant: table statistics are per-schema
-- pg_analyze must run for each tenant's tables separately
-- Set autovacuum per tenant if needed:
ALTER TABLE t_acme.tasks SET (autovacuum_vacuum_scale_factor = 0.01);
```

Interview Questions & Answers

Q1: What are the three main multi-tenant strategies in PostgreSQL?

A: (1) Shared schema with Row-Level Security (tenant_id column on all tables, RLS policies filter data), (2) Schema-per-tenant (each tenant has their own schema with identical tables), (3) Database-per-tenant (each tenant has their own PostgreSQL database). The choice depends on tenant count, isolation requirements, and operational complexity tolerance.

Q2: What is Row-Level Security (RLS) and how does it work?

A: RLS allows PostgreSQL to automatically filter rows based on the current user or session context. You define policies that are invisible WHERE clauses appended to every query. The application sets a session variable (e.g., SET app.current_tenant_id = 'uuid') and all queries automatically see only that tenant's data.

Q3: What is the difference between the `USING` and `WITH CHECK` clauses in an RLS policy?

A: `USING` filters rows on reads (SELECT, UPDATE, DELETE — what rows you can see). `WITH CHECK` restricts what rows you can write (INSERT, UPDATE — what `tenant_id` you can assign). Without `WITH CHECK`, a user could insert a row with another tenant's ID even if they can't read that tenant's data.

Q4: When would schema-per-tenant be preferred over shared schema?

A: When tenants need strong logical isolation, have compliance requirements (GDPR "right to be forgotten" is easier with per-schema dumps), need per-tenant customization (different table structures), or when the number of tenants is manageable (under ~5000) and cross-tenant reporting is rare.

Q5: How do you run schema migrations across all tenant schemas?

A: Write a migration script that queries `pg_namespace` for all tenant schemas, then iterates through each one and applies the migration in a transaction. Example: `SELECT nsname FROM pg_namespace WHERE nsname LIKE 't_%' AND nsname != 't_template'` then for each: `SET search_path = t_<slug>; ALTER TABLE tasks ADD COLUMN priority SMALLINT DEFAULT 2;`

Q6: What is the "noisy neighbor" problem in shared schema multi-tenancy?

A: One large tenant running expensive queries can consume resources that slow down other tenants sharing the same PostgreSQL instance. Mitigations: query timeouts per role (`ALTER ROLE tenant_role SET statement_timeout = '30s'`), connection limits per role, separate read replicas for heavy tenants, and resource groups (PostgreSQL 16+ resource management).

Exercises with Solutions

Exercise 1

Implement an RLS policy that allows tenant users to see all tasks in their tenant, but only allows them to update tasks they are assigned to.

Solution:

```
-- Allow SELECT for all tenant tasks
CREATE POLICY tasks_tenant_select ON tasks
  FOR SELECT TO saas_app_user
  USING (tenant_id = current_setting('app.current_tenant_id', TRUE)::UUID);

-- Allow UPDATE only for assigned tasks
CREATE POLICY tasks_update_assigned ON tasks
  FOR UPDATE TO saas_app_user
  USING (
    tenant_id = current_setting('app.current_tenant_id', TRUE)::UUID
    AND assignee_user_id = current_setting('app.current_user_id', TRUE)::UUID
  );

-- Allow INSERT for all tenant members
CREATE POLICY tasks_insert ON tasks
  FOR INSERT TO saas_app_user
  WITH CHECK (tenant_id = current_setting('app.current_tenant_id', TRUE)::UUID);
```

Cross-References

- `05_schema_design_ecommerce.md` — multi-tenant considerations in product catalogs
- `10_design_patterns.md` — tenant ID in audit trails
- `../10_schemas_namespacing.md` — schema-per-tenant implementation details
- `../14_Security/` — Row-Level Security deep dive
- `../12_Production_PostgreSQL/` — managing many schemas